SpecialistRoute Handover — 3. Data

Oct 25, 2026 · @Anas Alsirafy

The v2 dataset keeps the official Kaggle PlantDoc test set (236 images) untouched, and splits the cleaned training data into 1,897 train and 209 validation images. All images are CLAHE-processed and stored at 224×224.

Final splits and counts

The data is the Kaggle release of PlantDoc: 27 disease/healthy classes across 13 species, photographed in the field. Its original split is 2,316 train and 236 test images.

Split	Images	How it was made
Train	1,897	Original train after earlier quality cleaning and v2 leakage exclusion, minus the validation split
Validation	209	10% of cleaned train, stratified by class, look-alike pairs kept together, seed 42
Test	236	Official Kaggle test set, complete (includes 20 restored images)

Per component (the classifier sees every image, labelled by species):

Component	Classes	Train	Val	Test
Classifier (router)	7 species groups	1,897	209	236
Tomato	8	516	57	69
Rest	7	541	59	59
Corn	3	292	32	26
Apple	3	204	22	29
Potato	2	161	18	16
Bell pepper	2	92	10	17
Grape	2	91	11	20

Rest groups seven species that each have a single class: Blueberry, Cherry, Peach, Raspberry, Soyabean, Squash and Strawberry. Validation sets are small (10–59 images), so one image moves a component's validation accuracy by 2–10 points.

Leakage removal and duplicates

Duplicates were found on the **original** Kaggle images, and only training images were ever removed; the test set was never edited. The work is done by `prepare_final_dataset.py`, step 1.

1. **Find candidates:** exact duplicates by MD5, and near-duplicates by perceptual hash (pHash, Hamming distance ⩽ 8 of 64 bits).
2. **Review by hand:** every near-duplicate pair gets a side-by-side image and a `y` / `n` answer in `reports/near_duplicate_review.csv`. The script stops until every pair is answered.
3. **Apply the rules to each group of confirmed duplicates:**

Group contains	Action
A test image	Remove every **train** member (leak into test)
Train images of one class only	Keep one copy (largest resolution), remove the rest
Train images of several classes	Remove all (the label is ambiguous)
Several test images	Flag only, reported as a limitation (9 flags)

Result: **104 training images excluded**, listed with reasons in `reports/exclusion_list.csv`. An earlier audit also found 24 cross-class duplicates.

Kaggle PlantDoc reuses the same file names across splits and classes, so leakage checks must compare **content** (MD5), never file names. `check_protocol()` in the main notebook does this on every run.

Validation split and restored test images

Validation. 10% of each class's cleaned training images, seed 42, taken in whole groups. Near-duplicate pairs reviewed as "similar but not the same photo" (`n`) are kept together, so look-alike images never sit on both sides of the train/validation boundary. Validation is used for checkpoint selection and every design decision; test is used only for final reporting.

Restored test images. The v1 dataset had 20 test images removed during cleaning. Those removals used the same criteria as the training-set cleaning and were made without looking at model predictions. v2 restores all 20 so the test set is the complete official 236:

- Each restored image is rebuilt from its original with an exact replica of the CLAHE pipeline. A fidelity check re-processes existing test images and compares pixels.
- `verify_clahe_dataset.py` re-runs CLAHE on every original and compares it with the final dataset. Result: mean difference 0.0000, 0 images flagged.
- `reports/manifest.csv` marks them with status `restored from original (resize + CLAHE)`. Section 14 of the main notebook uses this to report the 216 original and 20 restored images separately.

The 20 restored images are much harder than the rest: about 38% accuracy, against 76% on the other 216 (Results / Findings).

Preprocessing

CLAHE is applied **once, offline**; images on disk are already processed, so `use_clahe` stays `False` in the config.

1. Resize the original to 224×224.
2. Convert BGR → LAB.
3. Apply CLAHE to the L channel only (clip limit 2.0, 8×8 tiles).
4. Convert back to BGR and save.

At load time images are upscaled from 224 to the model's size (384 for every v2 component) with TensorFlow: `tf.io.read_file` → `decode_image` → `tf.image.resize(..., 'bilinear')`. This matches `image_dataset_from_directory` pixel for pixel, so training and every evaluation see identical inputs.

- Never load model inputs with PIL. Its bicubic resize changed 13–18 test predictions in v1 (Known Issues).
- ConvNeXt's `preprocess_input` is a no-op in Keras; normalisation happens inside the model.
- The data-prep script's `img_size` must be **224**, not 384. With 384 the fidelity check fails.

Derived folders and report files

Derived folders (`Datasets/derived_v2/`) are built automatically by the main notebook and checked against their source on every use:

- `filtered_<keywords>__<hash>/{train,val,test}` : one per specialist, holding only that component's class folders. The hash comes from the source paths, so a different source never reuses an old folder.
- `crop_classifier_7class/{train,val,test}` : the router's data. Images are copied into one folder per species (six crops plus `Rest`) and renamed `<class>__<file>` , so names never collide.
- If a source changes, the check raises a **STALE folder** error. Delete the named folder and re-run; it is rebuilt from the current dataset.

Report files (`Datasets/reports/` , written by `prepare_final_dataset.py`):

File	Contents
`manifest.csv`	Every image: original split, final split, status (kept, restored, excluded)
`counts.csv`	Images per class in train, val and test
`exclusion_list.csv`	The 104 excluded training images, with reason and duplicate group
`duplicate_flags.csv`	Groups needing a mention in the paper (duplicates inside test, label conflicts with test)
`near_duplicate_review.csv`	Every pHash candidate pair with your `y` / `n` decision
`moved_files.csv`	Images found under a different class than in the original (possible relabels)